

# C++ INTERVIEW PREPARATION

*Simulation • Game Development • Unreal Engine • Technical Artist*

**Junior | Medior | Senior**

● MUST KNOW   ● IMPORTANT   ● BONUS

Version 1.0 • 2026

# SECTION 1 — Level Classification

*Each level builds on the previous. Percentages represent the share of core topics a candidate must confidently answer during an interview.*

| Level  | Experience | Required %          | Focus                                                  |
|--------|------------|---------------------|--------------------------------------------------------|
| Junior | 0–2 years  | 80% of core topics  | C++ fundamentals, basic OOP, STL basics, Unreal basics |
| Medior | 3–6 years  | 90% of core topics  | Modern C++, design patterns, UE systems, optimization  |
| Senior | 7+ years   | 95%+ incl. advanced | Engine internals, rendering, leadership, architecture  |

## JUNIOR (0–2 years)

### ABSOLUTELY REQUIRED:

- C++ basics: pointers, references, memory (stack vs heap)
- OOP: classes, inheritance, virtual methods, destructors
- STL: `std::vector`, `std::string`, basic iteration
- RAII and smart pointers (`unique_ptr` at minimum)
- Unreal: Actors, Components, Blueprints basics
- Can read and modify existing C++ code in Unreal

### NICE TO HAVE (Bonus):

- Move semantics / `std::move`
- Lambda basics
- Unreal reflection macros (`UCLASS`, `UPROPERTY`)

## MEDIOR (3–6 years)

### ABSOLUTELY REQUIRED:

- All Junior topics at expert level
- Modern C++11/14/17: move semantics, lambdas, auto, structured bindings
- Templates: function & class templates, specialization
- STL algorithms, unordered containers, iterators
- UObject lifecycle, Garbage Collection, delegates, timers
- Reflection system (`UCLASS`, `UPROPERTY`, `UFUNCTION`)
- Performance awareness: draw calls, LOD, profiling basics

### NICE TO HAVE (Bonus):

- `constexpr`, `SFINAE` basics
- C++20 ranges, concepts

- Unreal subsystems, asset manager

## SENIOR (7+ years)

### **ABSOLUTELY REQUIRED:**

- All Medior topics at expert level
- Template metaprogramming, CRTP, type traits
- Lock-free programming, memory ordering, threading
- Unreal rendering pipeline, custom passes, materials
- Engine extension: custom systems, editor tools, plugins
- Profiling with Unreal Insights, RenderDoc, Perf
- Mentoring, architecture decisions, technical leadership

### **NICE TO HAVE (Bonus):**

- Vulkan / DX12 knowledge
- Custom memory allocators
- Formal safety standards (MISRA, ISO 26262)

# SECTION 2 — C++ Interview Questions

---

## JUNIOR LEVEL

### Core Language

#### struct vs class

● **MUST KNOW** What is the difference between struct and class in C++?

→ Default access: struct members are public, class members are private. Both support OOP features identically. Convention: use struct for plain data, class for objects with behavior.

● **IMPORTANT** When would you use a struct over a class in a game engine?

→ For simple POD-like data: position (Vector3), color (RGBA), vertex data. No private invariants needed, just grouped fields.

#### Pointer vs Reference

● **MUST KNOW** What is the difference between a pointer and a reference?

→ A pointer holds an address and can be null or reassigned. A reference is an alias for an existing variable — cannot be null and cannot be resealed. Reference syntax is cleaner; use pointers when nullability or reassignment is needed.

● **MUST KNOW** Can a reference be null?

→ No. A reference must be bound to a valid object at initialization. Attempting to make a null reference is undefined behavior.

● **IMPORTANT** When do you pass by pointer vs by reference?

→ Pass by reference when the argument must exist. Pass by pointer when null is a valid state (optional parameter) or when ownership transfer is implied.

#### const vs constexpr

● **MUST KNOW** What does const mean in C++?

→ const promises that the value will not be modified after initialization. It is evaluated at runtime unless the compiler can optimize it away.

● **IMPORTANT** What is constexpr?

→ constexpr guarantees evaluation at compile time when all inputs are known. constexpr variables are implicitly const. Use constexpr for mathematical constants, array sizes, and compile-time computations.

● **BONUS** What is if constexpr (C++17)?

→ A compile-time conditional that discards the non-taken branch — enables conditional template code without SFINAE.

#### static Variables

● **MUST KNOW** What does static mean on a local variable?

→ The variable is initialized once and persists for the program's lifetime, shared across all calls to that function.

● **MUST KNOW** What does static mean on a class member?

→ The member belongs to the class, not to any instance. All instances share the same value.

● **IMPORTANT** What does static mean on a global variable or free function?

→ Limits the linkage to the current translation unit — it won't be visible to other .cpp files.

## Copy Constructor / Move Semantics / std::move

● **MUST KNOW** What is a copy constructor?

→ A special constructor that creates a new object as a copy of an existing one: MyClass(const MyClass& other). If not provided, the compiler generates a member-wise copy.

● **MUST KNOW** What is std::move?

→ std::move casts an lvalue to an rvalue reference, enabling the move constructor/assignment operator to transfer resources instead of copying them — typically O(1) vs O(n).

● **IMPORTANT** What is the Rule of Five?

→ If you define any of: destructor, copy ctor, copy assignment, move ctor, move assignment — you should define all five. Modern guideline: Rule of Zero (use smart pointers and let the compiler generate everything).

## RAII

● **MUST KNOW** Explain RAII with a real example.

→ Resource Acquisition Is Initialization: tie a resource's lifetime to an object's lifetime. Example: std::unique\_ptr acquires a heap allocation in its constructor and releases it in its destructor — even if an exception is thrown. Same pattern: std::lock\_guard locks a mutex on construction, unlocks on destruction.

● **IMPORTANT** Why is RAII critical in game engines?

→ Game objects are created and destroyed constantly. RAII prevents leaks under exceptions or early returns, and makes resource management predictable and automatic.

## Templates

● **MUST KNOW** What is a function template? Write an example.

→ template<typename T> T max(T a, T b) { return a > b ? a : b; } — the compiler generates a concrete version for each T used at the call site.

● **IMPORTANT** What is template specialization?

→ Providing a custom implementation for a specific type: template<> void print<bool>(bool v) { std::cout << (v ? "true" : "false"); }

● **BONUS** What is a variadic template?

→ A template accepting any number of type parameters: template<typename... Args>. Used by std::tuple, std::make\_unique, logging utilities.

## OOP

### Polymorphism

● **MUST KNOW** What is runtime polymorphism?

→ Selecting the correct function at runtime via virtual dispatch (vtable). Base pointer or reference calls the overriding derived method.

● **MUST KNOW** What is compile-time polymorphism?

→ Achieved via function overloading and templates — resolved entirely by the compiler with no vtable overhead.

● **IMPORTANT** What is the cost of virtual dispatch?

→ An indirect function call through a pointer (vtable lookup). Usually negligible, but in tight inner loops with millions of entities it can cause cache misses.

## Virtual Methods

● **MUST KNOW** What does the virtual keyword do?

→ Marks a method for dynamic dispatch. When called through a base pointer/reference, the most-derived override is called.

● **MUST KNOW** What does override do (C++11)?

→ Tells the compiler this method is intended to override a base virtual. Compile error if no matching base virtual exists — prevents typo bugs.

## Virtual Destructors

● **MUST KNOW** When must a destructor be virtual?

→ Whenever objects of a derived class may be deleted through a base class pointer. Without a virtual destructor, only the base destructor runs — resource leak in the derived part.

● **MUST KNOW** Does every class need a virtual destructor?

→ No. Only classes designed to be used polymorphically as a base. Final classes or value types (structs) do not need one.

## STL

### std::vector vs std::array

● **MUST KNOW** What is the main difference between std::vector and std::array?

→ std::array has a fixed compile-time size stored on the stack. std::vector is heap-allocated and can resize. For fixed-size small buffers (vertex buffer, transform array) prefer std::array.

● **IMPORTANT** When does std::vector reallocate?

→ When size() == capacity(). It typically doubles capacity —  $O(1)$  amortized push\_back, but triggers element copies/moves and invalidates all iterators.

### std::map vs std::unordered\_map

● **MUST KNOW** What is the time complexity difference?

→ std::map:  $O(\log n)$  find/insert (red-black tree, ordered). std::unordered\_map:  $O(1)$  average (hash table, unordered). For lookups by string key in an asset manager, unordered\_map is faster.

● **IMPORTANT** When would you prefer std::map?

→ When you need ordered iteration, range queries, or your key type has no suitable hash function.

## Iterators

### ● **MUST KNOW** What is an iterator?

→ An object that abstracts traversal over a container. It supports at minimum: `operator*`, `operator++`, and comparison. Enables STL algorithms to work on any container.

### ● **IMPORTANT** What invalidates iterators?

→ `std::vector`: any reallocation (`push_back` past capacity), `erase/insert` before the iterator.  
`std::map`: only `erase` of the pointed-to element.

## `std::sort`

### ● **MUST KNOW** How do you sort a vector of floats?

→ `std::sort(v.begin(), v.end());`; —  $O(n \log n)$  introsort. For descending: `std::sort(v.begin(), v.end(), std::greater<float>());`;

### ● **IMPORTANT** How do you sort a vector of structs by a field?

→ Use a lambda: `std::sort(actors.begin(), actors.end(), [](const Actor& a, const Actor& b){ return a.priority > b.priority; });`;

## Modern C++

### Lambdas

### ● **MUST KNOW** Write a lambda that captures `x` by value and doubles it.

→ `int x = 5; auto f = [x]() { return x * 2; };` — captures `x` by copy. Use `[&x]` to capture by reference.

### ● **MUST KNOW** What are common use cases for lambdas in game code?

→ Callbacks for delegates/events, predicates for `std::sort/std::find_if`, deferred actions in timers, UI button handlers.

### ● **IMPORTANT** What is a generic lambda (C++14)?

→ `auto add = [](auto a, auto b){ return a + b; };` — the compiler generates a template for each combination of argument types.

### `auto` / `decltype`

### ● **MUST KNOW** When should you use `auto`?

→ When the type is obvious (iterator, lambda, complex template return type) or when changing the type later should propagate automatically. Avoid `auto` where it hides important type information.

### ● **IMPORTANT** What is `decltype`?

→ Deduces the type of an expression without evaluating it: `decltype(a + b)` — useful in trailing return types and template metaprogramming.

### Structured Bindings (C++17)

### ● **IMPORTANT** What are structured bindings?

→ `auto [key, value] = *map.find("hp");` — unpacks a pair, tuple, or aggregate into named variables. Improves readability in range-for loops over maps.

### ● **BONUS** Can you use structured bindings with custom structs?

→ Yes, if the struct has all public members or you provide a `get<>` specialization (same mechanism as `std::tuple`).

## Ranges (C++20)

### ● **BONUS** What is the main advantage of C++20 ranges?

→ Composable, lazy, pipeline-style algorithms: `auto names = actors | std::views::filter(isAlive) | std::views::transform(&Actor::getName);` — no temporaries, readable left-to-right.

## Memory & Performance

### Stack vs Heap

#### ● **MUST KNOW** What is stored on the stack vs heap?

→ Stack: local variables, function parameters, return addresses — fast LIFO allocation, limited size (~1–8 MB). Heap: dynamic allocations (`new/delete`, `malloc`) — large, slow, manual or GC managed.

#### ● **MUST KNOW** What happens when the stack overflows?

→ Crash / segmentation fault. Caused by deep or infinite recursion, or very large local arrays.

### Memory Leaks

#### ● **MUST KNOW** What is a memory leak?

→ Heap memory allocated with `new` but never released with `delete` — the process consumes more and more RAM until it crashes or is killed. Detected by tools like `AddressSanitizer` or `Valgrind`.

#### ● **MUST KNOW** How do smart pointers prevent memory leaks?

→ `unique_ptr`'s destructor calls `delete` automatically. `shared_ptr` uses reference counting — object is deleted when the last owner goes out of scope.

### Object Lifetime

#### ● **MUST KNOW** In what order are destructors called for class members?

→ Reverse declaration order — the last declared member is destroyed first. Base class destructors run after all derived members.

#### ● **IMPORTANT** What is the lifetime of a static local variable?

→ Initialized on first function call, destroyed at program exit. Thread-safe in C++11 (magic static).

## MEDIOR LEVEL

### Core Language — Deeper

● **MUST KNOW** Explain move semantics in detail. When is a move constructor called?

→ When a temporary (rvalue) is passed or returned. The move ctor transfers ownership of resources (heap pointer, file handle) from the source — leaving it in a valid but unspecified state — instead of copying. Avoids  $O(n)$  copies for large containers.

● **MUST KNOW** What is perfect forwarding and why is it needed?

→ `template<typename T> void wrapper(T&& arg) { target(std::forward<T>(arg)); }` — `std::forward` preserves the value category (lvalue stays lvalue, rvalue stays rvalue). Without it, the arg inside wrapper is always an lvalue.

● **MUST KNOW** What is SFINAE?

→ Substitution Failure Is Not An Error. When template argument deduction fails for a candidate, that overload is silently ignored rather than producing a compiler error. Enables conditional template enablement based on type traits.

● **MUST KNOW** What are type traits? Give examples.

→ Compile-time type predicates and transformations from `<type_traits>`: `std::is_integral<T>`, `std::is_pointer<T>`, `std::remove_reference<T>`, `std::enable_if<condition, T>`. Used heavily in generic library code.

● **IMPORTANT** What is constexpr vs consteval (C++20)?

→ `constexpr` functions may run at compile or runtime. `constexpr` functions (immediate functions) must run at compile time — a compile error if called with runtime arguments.

● **IMPORTANT** What is `std::string_view` and when should you use it?

→ A non-owning read-only view over a string buffer. Prefer over `const std::string&` for function parameters — avoids allocation when passing string literals or substrings.

● **BONUS** What is CRTP (Curiously Recurring Template Pattern)?

→ `class Derived : public Base<Derived> {};` — enables static polymorphism. Base can call Derived methods at compile time via `static_cast<Derived*>(this)`, avoiding vtable overhead.

### OOP — Deeper

● **MUST KNOW** What is the diamond inheritance problem?

→ When two classes both inherit from a common base, and a fourth inherits from both — two copies of the base exist. Solution: virtual inheritance (`class B : virtual public A`).

● **MUST KNOW** What is object slicing?

→ Assigning a derived object to a base object by value — the derived-specific data is sliced off. Fix: always store polymorphic objects by pointer or reference.

● **IMPORTANT** Compare virtual dispatch vs function pointer vs `std::function`.

→ Virtual: zero overhead per call site, vtable indirection. Function pointer: same cost, no type safety. `std::function`: type-erased, allocates for large callables, slowest — use for callbacks stored by value.

## STL — Deeper

● **MUST KNOW** What is the complexity of `std::unordered_map` in the worst case?

→  $O(n)$  — all keys hash to the same bucket. In practice  $O(1)$  average. Worst case occurs with a poor hash function or a hash-collision attack.

● **MUST KNOW** What is `std::move_iterator`?

→ An iterator adaptor that converts dereferencing into an rvalue — enables move semantics in algorithms: `std::vector<T> v2(std::make_move_iterator(v1.begin()), std::make_move_iterator(v1.end()));`

● **IMPORTANT** What are the five iterator categories?

→ Input, Output, Forward, Bidirectional, Random Access (+ Contiguous in C++20). Algorithms require minimum categories — e.g. `std::sort` requires `RandomAccess`.

● **BONUS** What is `std::span` (C++20)?

→ A non-owning view over a contiguous sequence — like `string_view` but for any type. Prefer over raw pointer + size pairs.

## Modern C++ — Deeper

● **MUST KNOW** Explain `std::optional` and when to use it.

→ A value that may or may not be present — safer alternative to returning -1 or null. `std::optional<int> find()` — caller checks with `has_value()` or operator `bool` before accessing.

● **MUST KNOW** What is `std::variant`?

→ A type-safe union. `std::variant<int, float, std::string>` holds exactly one of its types. Access via `std::get<T>` or `std::visit` with a visitor.

● **IMPORTANT** What is `std::any`?

→ Type-erased container for any copyable type. Slower than `variant` (heap allocation for large types). Use only when the set of types is truly open.

● **BONUS** What are C++20 concepts?

→ Named compile-time constraints on template parameters: `template<std::integral T> void foo(T x);` — clearer error messages and better overload resolution than SFINAE.

## Memory & Performance — Deeper

● **MUST KNOW** What is cache locality and why does it matter for simulation/game code?

→ Modern CPUs fetch 64-byte cache lines. Iterating over contiguous data (`std::vector`) keeps the cache hot. Pointer-chasing (linked list, polymorphic objects in random heap locations) triggers cache misses — 100x slower. Prefer SoA (Structure of Arrays) over AoS for hot loops.

● **MUST KNOW** What is a memory pool allocator?

→ Pre-allocates a block of memory and serves requests from it. Zero calls to `malloc` in the real-time loop — deterministic allocation latency, no fragmentation. Essential for real-time simulations.

● **IMPORTANT** What is false sharing?

→ Two threads modify different variables that happen to share a cache line — each write invalidates the other core's cache. Fix: pad data structures so each thread's hot data is on separate cache lines (`alignas(64)`).

● **IMPORTANT** What tools detect memory bugs in C++?

→ AddressSanitizer (out-of-bounds, use-after-free), MemorySanitizer (uninitialized reads), LeakSanitizer (leaks), Valgrind Memcheck. Enable with `-fsanitize=address` in GCC/Clang.

## SENIOR LEVEL

### Template Metaprogramming

- **MUST KNOW** What is template metaprogramming (TMP)?

→ Using templates to perform computation at compile time. Example: compile-time factorial via recursive template specialization. Modern alternative: constexpr functions (clearer, C++11+).

- **MUST KNOW** Explain `std::enable_if` and a modern replacement.

→ `std::enable_if<condition>::type` causes substitution failure when condition is false, removing the overload. Modern replacement: `if constexpr` (C++17) or `requires` clause / `concepts` (C++20).

- **IMPORTANT** What is tag dispatching?

→ Using empty tag types to select overloads at compile time based on type traits — avoids SFINAE complexity. Largely superseded by `if constexpr` and `concepts`.

- **BONUS** Explain expression templates with a use case.

→ Delay evaluation of expression trees to avoid temporaries. Eigen's matrix library uses them: `Matrix result = a + b + c;` builds an expression tree, evaluating into result in a single loop pass.

### Concurrency

- **MUST KNOW** What is a data race?

→ Two threads access the same memory simultaneously where at least one writes, without synchronization. Undefined behavior in C++. Detected by ThreadSanitizer (`-fsanitize=thread`).

- **MUST KNOW** What are `std::atomic` memory ordering options?

→ `relaxed` (no ordering guarantees), `acquire` (loads see prior releases), `release` (stores visible before later acquires), `acq_rel`, `seq_cst` (full sequential consistency — safest, default, but most expensive).

- **MUST KNOW** What is a lock-free data structure?

→ Uses atomic compare-and-swap (CAS) instead of mutexes — threads never block, only retry. Examples: lock-free stack, SPSC queue. Used for real-time simulation event queues.

- **IMPORTANT** What is a thread pool and how would you implement one?

→ A fixed number of worker threads pulling tasks from a shared queue. Workers block on a `condition_variable` when idle. Submitted tasks are pushed to the queue and a thread is notified. TBB or `std::async` are production alternatives.

- **IMPORTANT** What is priority inversion?

→ A low-priority thread holds a lock needed by a high-priority thread — effectively running at low priority. Solution: priority inheritance mutex (available on POSIX with `PTHREAD_PRIO_INHERIT`).

### Advanced Performance

- **MUST KNOW** What is SIMD and when do you use it in simulation?

→ Single Instruction Multiple Data — one CPU instruction operates on 4/8/16 floats simultaneously (SSE/AVX). Used for particle updates, physics collision tests, audio mixing. Libraries: Eigen (auto-vectorized), DirectXMath, or manual intrinsics.

● **MUST KNOW Explain Profile-Guided Optimization (PGO).**

→ Compile with profiling instrumentation, run representative workloads, then recompile using the profile data. Compiler optimizes branch prediction, inlining, and code layout for actual runtime patterns. Gains of 10–20% common.

● **IMPORTANT What is the Roofline model?**

→ A performance model showing whether code is compute-bound (limited by FLOPS) or memory-bound (limited by bandwidth). Determines whether to optimize computation or memory access patterns.

## SECTION 3 — Bonus Questions Per Level

### JUNIOR — Bonus & Trick Questions

- **MUST KNOW** What is the output of: `int x = 0; std::cout << x++ << " " << ++x;`  
→ Undefined behavior — the order of evaluation of function arguments is unspecified in C++. Interviewers use this to test awareness of UB. Correct answer: 'this is undefined behavior.'
- **IMPORTANT** Is `std::vector` contiguous in memory?  
→ Yes. Guaranteed since C++03. Elements are contiguous — `&v[0]` is a valid pointer to the first element. This is why it outperforms `std::list` for cache performance.
- **IMPORTANT** Can you have a virtual constructor?  
→ No. Constructors cannot be virtual because the object's dynamic type is not yet established when the constructor runs. Use factory functions or the Abstract Factory pattern instead.
- **BONUS** Scenario: Your game crashes intermittently with a segfault. How do you start debugging?  
→ Run under AddressSanitizer to catch memory errors. Use a debug build with symbols. Check for use-after-free (common culprit). Examine the stack trace — the crash address often reveals the bad pointer. Run in Valgrind if ASan doesn't catch it.

### MEDIOR — Bonus & Trick Questions

- **MUST KNOW** Does `std::shared_ptr` make your code thread-safe?  
→ Only the reference count is thread-safe. The pointed-to object is NOT protected. Two threads can safely hold `shared_ptr`s to the same object, but must externally synchronize reads/writes to the object.
- **MUST KNOW** What is the most important thing to check before calling `std::sort`?  
→ That the comparator is a strict weak ordering (irreflexive, asymmetric, transitive). A broken comparator causes undefined behavior — often a crash or infinite loop.
- **IMPORTANT** Scenario: You profile the game and see 80% of frame time in one physics update loop. What do you investigate first?  
→ Cache misses (check Cachegrind / VTune). Are you iterating over an AoS layout instead of SoA? Are there branchy virtual calls inside the loop? Are you allocating inside the loop? Profile before optimizing.
- **BONUS** What is the as-if rule?  
→ The compiler can transform code in any way that doesn't change the observable behavior of the program — return values, I/O, volatile reads. This allows aggressive inlining, constant folding, dead code elimination.

### SENIOR — Bonus & Trick Questions

● **MUST KNOW** Scenario: You need to serialize 10,000 physics objects to disk every frame at 60Hz. How do you design this?

→ Avoid per-object serialization. Use a flat binary buffer matching memory layout (SoA). Memory-map the file (mmap) or use async I/O. Delta-encode if objects rarely change. Consider a separate serialization thread with double-buffering.

● **MUST KNOW** When would you choose to not use exceptions?

→ In real-time simulation hot loops, on embedded/console platforms with disabled exceptions (-fno-exceptions), in safety-critical code (MISRA). Use std::expected (C++23) or error codes instead.

● **IMPORTANT** Scenario: Two senior engineers disagree on whether to use ECS or OOP for a new simulation module. How do you arbitrate?

→ Profile first — is there a measured performance problem? ECS wins for cache-friendly bulk operations on thousands of entities. OOP wins for complex single-entity behavior with many virtual methods. Often the answer is a hybrid. Decision must be data-driven.

● **BONUS** What is the ABI stability problem and why does it matter for a plugin system?

→ Changing a class layout (adding a member, changing virtual method order) breaks binary compatibility with plugins compiled against the old ABI. Solutions: C interface for plugins, opaque pointers, or versioned interfaces.

# SECTION 4 — Unreal Engine & Technical Artist

*This section covers Unreal-specific C++ knowledge, common interview questions, and Technical Artist responsibilities spanning shaders, tools, and optimization.*

## UE JUNIOR — Must Know

### Architecture Basics

● **MUST KNOW** What is an Actor in Unreal Engine?

→ An Actor is the base class for any object that can be placed in a level. It has a transform (location, rotation, scale) and can contain Components. AActor is the C++ base class.

● **MUST KNOW** What is a Component?

→ A modular piece of functionality attached to an Actor (e.g. UStaticMeshComponent for a 3D mesh, UCameraComponent). Components follow composition-over-inheritance.

● **MUST KNOW** What is the gameplay lifecycle order?

→ PostInitializeComponents → BeginPlay → Tick → EndPlay → BeginDestroy. Understand where to initialize vs where to start game logic.

● **MUST KNOW** What is a Blueprint?

→ Unreal's visual scripting system — a graph of nodes representing events and function calls. Compiles to bytecode interpreted by the Blueprint VM. C++ classes can expose functions and properties to Blueprint via macros.

● **MUST KNOW** How do you expose a C++ variable to Blueprint?

→ UPROPERTY(EditAnywhere, BlueprintReadWrite) float Speed = 500.f; — the UPROPERTY macro registers it with the reflection system.

● **IMPORTANT** What is the difference between EditAnywhere and VisibleAnywhere?

→ EditAnywhere: editable in both defaults and instances. VisibleAnywhere: read-only in editor. BlueprintReadWrite: accessible in Blueprint graphs.

● **IMPORTANT** How do you call a C++ function from Blueprint?

→ UFUNCTION(BlueprintCallable) void FireWeapon(); — makes the function available as a Blueprint node.

● **BONUS** What is a GameMode?

→ AGameModeBase controls the rules of the game — which player controller, pawn, and HUD classes to use. Only exists on the server in multiplayer.

## UE MEDIOR — Must Know

### UObject Lifecycle & Memory

● **MUST KNOW** What is UObject?

→ The base class for almost everything in Unreal that needs reflection, garbage collection, serialization, or replication. All UCLASS types inherit from UObject (directly or via AActor, UActorComponent, etc.).

● **MUST KNOW** How does Unreal's Garbage Collector work?

→ Mark-and-sweep GC running periodically. Any UObject reachable from a UPROPERTY or AddToRoot is kept alive. Objects stored in raw pointers (non-UPROPERTY) are invisible to the GC and may be collected — use UPROPERTY or TWeakObjectPtr.

● **MUST KNOW** What is the difference between TObjectPtr, TWeakObjectPtr, and TSharedPtr?

→ TObjectPtr: strong reference tracked by GC (preferred for UPROPERTY). TWeakObjectPtr: doesn't prevent GC — check IsValid() before use. TSharedPtr: reference-counted, for non-UObject C++ types only.

● **MUST KNOW** What are delegates in Unreal?

→ Type-safe callback wrappers: DECLARE\_DELEGATE, DECLARE\_MULTICAST\_DELEGATE. Multicast delegates broadcast to multiple bound functions — used for events (OnActorBeginOverlap, OnHealthChanged).

## Reflection System

● **MUST KNOW** What does the Unreal reflection system provide?

→ Runtime type information (GetClass(), IsA<T>()), property iteration, Blueprint exposure, serialization, and garbage collection integration. Generated by Unreal Header Tool (UHT) which reads UCLASS/UPROPERTY/UFUNCTION macros.

● **MUST KNOW** What is a UCLASS?

→ UCLASS() class UMyClass : public UObject { GENERATED\_BODY() ... }; — registers the class with the reflection system. GENERATED\_BODY() inserts UHT-generated boilerplate.

## Optimization Basics

● **MUST KNOW** What is a draw call and why does minimizing them matter?

→ A draw call is a CPU command to the GPU to render geometry. Each has driver overhead. Merging meshes, using instanced rendering, and reducing material complexity reduces draw call count — major mobile and console bottleneck.

● **MUST KNOW** What is LOD (Level of Detail)?

→ Using lower-poly meshes at distance. Unreal supports HLOD (Hierarchical LOD) and automatic LOD generation. Reduces GPU vertex processing cost.

● **IMPORTANT** What is Unreal Insights?

→ Unreal's built-in profiling tool — captures CPU traces (named events via TRACE\_CPUPROFILER\_EVENT\_SCOPE), GPU timings, memory, and network. Essential for finding frame budget overruns.

● **IMPORTANT** What is the Unreal stat system?

→ DECLARE\_CYCLE\_STAT macros instrument code blocks. Visible in Stat commands: stat game, stat engine, stat gpu. Used to identify slow systems.

● **BONUS** What is Nanite?

→ UE5's virtualized micropolygon geometry system — streams and renders only the triangles visible on screen regardless of source polygon count. Eliminates manual LOD for static meshes.

## UE SENIOR — Must Know

### Engine-Level Understanding

● **MUST KNOW** Explain the Unreal rendering pipeline at a high level.

→ Game Thread → Render Thread → RHI Thread → GPU. The game thread kicks off a render frame by enqueueing render commands. The render thread builds draw lists and submits to the RHI (Render Hardware Interface) thread which translates to D3D12/Vulkan/Metal commands for the GPU.

● **MUST KNOW** What is a Material in Unreal and how does it map to shaders?

→ A Material is a graph of Material Expressions compiled to HLSL shader code by the Material Editor. It can produce vertex, pixel, and compute shaders. Technical Artists author materials; engineers extend with Custom expressions or Material Functions.

● **MUST KNOW** What is a Render Pass / Scene Capture Component use case?

→ USceneCaptureComponent2D renders the scene from a given viewpoint to a RenderTarget — used for minimap, security cameras, reflections. Each capture is an additional draw call budget cost.

● **IMPORTANT** How do you create a custom Unreal editor tool?

→ Implement a UEditorUtilityWidget or an FEditorExtension. For complex tools: create a plugin with an FSlateApplication window, register it via IModuleInterface. Technical Artists commonly build mesh processing, batch rename, and LOD automation tools.

● **IMPORTANT** What is the Gameplay Ability System (GAS)?

→ A framework for abilities, attributes, effects, and tags. Used in complex games (MOBA, RPG). Technical Artists often configure Ability blueprints; engineers write UGameplayAbility subclasses in C++.

### Typical Unreal Interview Questions

● **MUST KNOW** What is the difference between ConstructorHelpers and AssetManager?

→ ConstructorHelpers::FObjectFinder loads an asset synchronously in the constructor — hard reference, always loaded. AssetManager / FSoftObjectPath enables async streaming — asset only loaded when needed. Prefer the latter in shipping games.

● **MUST KNOW** How do you avoid tick overhead?

→ Disable tick on actors that don't need it (PrimaryActorTick.bCanEverTick = false). Use timers (GetWorldTimerManager()) instead of tick for periodic checks. Profile with stat game to find expensive tickers.

● **IMPORTANT** What is Actor spawning and what parameters matter?

→ SpawnActor<T>(Class, Transform, SpawnParams). SpawnParams control collision handling (AlwaysSpawn vs AdjustIfPossibleButAlwaysSpawn), owner, instigator.

## ● BONUS What is Lumen?

→ UE5's fully dynamic global illumination and reflections system. Software and hardware ray tracing back-ends. Eliminates baked lightmaps for many use cases but has GPU cost — Technical Artists must tune its quality settings per platform.

## C++ in Unreal Context

*Unreal C++ adds conventions on top of standard C++ — understanding both is required.*

- Naming: U prefix for UObject subclasses, A for Actor, F for structs/non-UObject, I for interfaces, E for enums.
- No raw new/delete for UObjects — use `NewObject<T>()` and rely on GC. For non-UObject: use `MakeShared/TSharedPtr` or standard smart pointers.
- UPROPERTY prevents GC; raw `UObject*` can be collected at any time.
- Build system: Unreal Build Tool (UBT) with `.Build.cs` files. Modules are compilation units — keep modules focused to reduce rebuild times.
- Hot reload: Live Coding allows C++ changes without full restart — not all changes are supported (vtable changes, new UPROPERTYs require editor restart).

## SECTION 5 — Difficulty Color Coding Reference

---

| Label       | Meaning                                           | Impact on Interview                            |
|-------------|---------------------------------------------------|------------------------------------------------|
| ● MUST KNOW | Critical — getting this wrong fails the interview | Immediate disqualification at most companies   |
| ● IMPORTANT | Strong candidate — knowing this sets you apart    | Differentiates mid-tier from strong candidates |
| ● BONUS     | Senior differentiator or specialized knowledge    | Impresses, rarely required to pass             |

## SECTION 6 — Summary & Preparation Guide

---

### Required Knowledge Per Level

| Topic               | Junior               | Medior                   | Senior                |
|---------------------|----------------------|--------------------------|-----------------------|
| C++ Standard        | C++11 basics         | C++14/17 fluent          | C++20/23 + internals  |
| Pointers / Memory   | Basics, smart ptrs   | Custom alloc, lifetimes  | Lock-free, allocators |
| Templates           | Function templates   | SFINAE, specialization   | TMP, CRTP, concepts   |
| Concurrency         | Thread awareness     | mutex, atomic, futures   | Lock-free, wait-free  |
| OOP                 | Inheritance, virtual | Composition, slicing     | Type erasure, policy  |
| STL                 | vector, map, sort    | unordered, ranges        | Custom iterators      |
| Performance         | Big-O awareness      | Cache, profiling         | SIMD, GPU, NUMA       |
| UE Architecture     | Actors, Components   | UObject, GC, delegates   | Rendering pipeline    |
| UE Reflection       | UPROPERTY basics     | UHT, full macros         | Custom metadata       |
| UE Optimization     | Draw call concept    | LOD, stat commands       | Insights, RenderDoc   |
| Shaders / Materials | Awareness            | Material graph authoring | HLSL, custom passes   |
| Editor Tools        | Blueprint scripting  | Utility widgets          | Plugin development    |

### Progress Checklist

#### Junior Checklist

- ☐ I can explain pointers, references, and smart pointers clearly
- ☐ I understand stack vs heap and can spot memory leaks
- ☐ I know OOP: classes, inheritance, virtual methods, virtual destructors
- ☐ I can use `std::vector`, `std::map`, `std::sort` with lambdas
- ☐ I understand RAII and can give a concrete example
- ☐ I can write and read basic Unreal C++ (UCLASS, UPROPERTY, UFUNCTION)
- ☐ I know the Actor/Component model and gameplay lifecycle
- ☐ I can create simple Blueprints and call C++ functions from them

## Medior Checklist

- ☐ I understand move semantics and perfect forwarding
- ☐ I can write templates with specializations and SFINAE concepts
- ☐ I know `std::optional`, `std::variant`, structured bindings, ranges
- ☐ I understand `UObject` GC, `TObjectPtr`, `TWeakObjectPtr` correctly
- ☐ I can use delegates and multicast delegates in Unreal
- ☐ I know cache locality and how AoS vs SoA affects performance
- ☐ I can profile with Unreal Insights and explain findings
- ☐ I understand draw calls, LOD, and basic Material authoring

## Senior Checklist

- ☐ I can design systems from scratch, justify trade-offs
- ☐ I understand template metaprogramming and type traits deeply
- ☐ I know concurrency primitives and lock-free algorithm fundamentals
- ☐ I can explain the Unreal rendering pipeline (game thread to GPU)
- ☐ I've written or extended Unreal editor plugins or custom tools
- ☐ I can profile at GPU level with RenderDoc or similar tools
- ☐ I understand HLSL / Material compiler output
- ☐ I can mentor others and make architecture decisions with confidence

## How to Prepare — Practical Advice

*Quality of practice beats quantity. One project with real profiling feedback is worth more than 100 LeetCode solutions.*

### Step 1 — Solidify C++ fundamentals

- Read: Effective Modern C++ by Scott Meyers (most impactful C++ book)
- Read: C++ Concurrency in Action by Anthony Williams for threading
- Practice on godbolt.org to see assembly output of your code

### Step 2 — Build real Unreal projects

- Implement a feature entirely in C++ (no Blueprint) — custom movement, ability system
- Write an editor utility widget that automates a workflow task
- Profile a scene with Unreal Insights and optimize one bottleneck

### Step 3 — Review Unreal source code

- Read UE5 source on GitHub — study `UPrimitiveComponent`, `UActorComponent`, `FRenderCommandFence`
- Understand how `UPROPERTY` macros expand (generated `.generated.h` files)

### Step 4 — Mock interviews

- Explain every topic out loud — if you can't explain it simply, you don't know it
- Time yourself on coding questions — 20 minutes per problem max
- Review your answers against this guide after each mock session

**Recommended Resources:**

- [cppreference.com](http://cppreference.com) — C++ language reference
- [dev.epicgames.com/documentation](http://dev.epicgames.com/documentation) — official UE5 docs
- [unrealengine.com/tech-blog](http://unrealengine.com/tech-blog) — performance and architecture articles
- [godbolt.org](http://godbolt.org) — Compiler Explorer
- [quick-bench.com](http://quick-bench.com) — micro-benchmarking

**Good luck — ship great simulations.**